

Lecture 6: Deep Learning

Alex Reinhart
Machine Learning II: 46-927

March 1–8, 2021

Announcements

- ▶ Install PyTorch for HW6:
`https://pytorch.org/get-started/locally/`
- ▶ HW6 will be final HW, and will be short.
- ▶ Project status report on GradeScope, “due” Thursday
- ▶ Project reports due 11:59pm ET on Thursday, March 11th
- ▶ Project presentations 10am–1pm ET on Tuesday, March 16th

Today

Deep learning!

Deep learning

The intent of today's lecture is to introduce you to ideas from deep learning.

This is generally a large topic that is beyond the scope of this mini.

The hope is that you'll become comfortable with some of the ideas, goals, and language, so that you can

- ▶ Recognize when it is potentially appropriate
- ▶ Discuss the basic ideas
- ▶ More easily learn about it when you need to

Much of this material and several examples are drawn from the book *Deep Learning* by Goodfellow, Bengio, Courville. Available free online. Referred to as DLB in slides.

<https://www.deeplearningbook.org/>

Deep learning

Deep learning is one of the hottest current topics in machine learning.

It has roots in ideas from the 50–60s and from the 90s, but it's made a big resurgence and massive progress in the last 10 years.

Builds on a lot of the past work in neural networks, recently enabled by improvements in computation power, optimization tricks, and lots of impressive engineering work.

Major goals: flexible modeling of complex problems, ability to capture levels of abstraction and solve problems of feature engineering.

Getting started

Start by considering linear regression. We observe a bunch of features $x \in \mathbb{R}^p$ and outcomes $y \in \mathbb{R}$. We model them as a linear regression:

$$\hat{y} = f(x) = x\hat{\beta}$$

We find β by solving the following optimization problem:

$$\operatorname{argmin}_{\beta} \|Y - X\beta\|_2^2 = \operatorname{argmin}_{\beta} \sum_{i=1}^n (y_i - x_i^T \beta)^2$$

This works great when our regression function $\mathbb{E}(Y \mid X = x)$ is approximately linear. What if it's not?

Beyond linear regression

We've seen some approaches: we can move to a more complicated f function: random forests, bagging, SVM.

Suppose we want to improve linear regression instead. For many years, this problem was approached by engineering functions $\phi_j(x)$ to provide custom features.

$$y = \phi(x)^T \beta$$

In image recognition, edge detection, corner detection, keypoints (SIFT).

This is incredibly expensive! It lead to improvements, but at great cost. It's also very difficult to scale to new problems.

Maybe I can do it for handwriting and voice dictation, but what about identifying facial expressions, translating languages, colorizing images, locating cats, image captioning, etc.

Beyond linear regression

Deep learning tries to construct these ϕ from the data.

$$y = \phi(x; \theta)^T \beta = \sum_{j=1}^{p'} \phi_j(x; \theta_j) \beta_j$$

where $\theta \in \mathbb{R}^d$. If we can estimate these $\phi_j(x; \theta_j)$ well, then we've "replaced" feature engineering!

What should we use for $\phi_j(x; \theta_j)$? We like linear functions, what if we define $\phi_j(x; \theta_j) = x^T \theta_j$?

Why is this a bad idea?

Beyond linear regression

Ok, so we can't use a linear function. What's non-linear but similar to linear regression?

$$\phi_j(x; \theta_j) =$$

Common choices for g :

XOR example

So far

So now we have a model of the form

$$\begin{aligned} y &= \sum_{j=1}^{p'} w_j \phi(x; \theta_j, c_j) \\ &= \sum_{j=1}^{p'} w_j \max\{0, x^T \theta_j + c_j\} \end{aligned}$$

These $\max\{0, x^T \theta_j + c_j\}$ provide simple features of x to use in your final regression model.

Already there are some gains, see previous XOR example.

How can we get more complicated features?

First, let's think about what we've done.

$$y = \sum_{j=1}^{p'} w_j \phi(x; \theta_j, c_j) = \sum_{j=1}^{p'} w_j \max\{0, x^T \theta_j + c_j\}$$

We can illustrate this as:

This suggests a path to more complex features.

More complexity

To get more complex features, we can just recurse! Build more layers of features, each of which is a non-linear function of a linear transformation of the previous.

This is the basic idea of *feed-forward neural networks* or *multilayer perceptrons*.

Feed-forward neural networks

This is a very flexible structure. It was shown (1989!) that feed-forward networks with a single hidden layer can approximate functions to any desired precision, given enough hidden units!

Of course, this doesn't say that you can find such a network...

In practice, it's hard to find such a single layer network. The abstraction of having multiple layers makes this simpler.

In general

Outputs

We can stick a bunch of final functions on top to get different outputs. Let h be the output of the final layer.

- ▶ Continuous outcome: just use a $w^T h + b$ linear function!
- ▶ Binary outcome: Just use logistic

$$\frac{1}{1 + e^{-w^T h}} =$$

- ▶ Multiple categories: soft-max

$$\frac{e^{w_i^T h}}{\sum_j e^{w_j^T h}}$$

- ▶ Other much more complicated outputs: All sorts of conditional distributions for y given x : Gaussians with interesting covariance matrices, mixture models of various sorts.

Network architecture

As we will see later, much of what makes neural networks difficult is deciding their **architecture**: how many layers? How are they connected? What types of functions do they use? What is our output layer?

In dimension reduction, we often adjusted the reduction method to suit the data type, with carefully chosen distances and spaces. In neural nets, we pick layer types and architectures best suited for the data we have.

Networks beyond feed-forward fully-connected networks are often designed for specific data types: convolutional neural networks, recurrent neural networks...

Now what? How do we find the weights?

Suppose we decide on an architecture: connections, numbers of nodes, transformations.

How do we actually find the weights?

Just like every other time, we define a loss and minimize it. It's just a bit more complicated now.

Loss function

Our loss functions tend to look very similar.

For continuous outputs, we might look $\|y - \hat{y}\|_2^2$.

For binary or multiple category outputs, we might look at the log likelihood of y_i :

$$\text{Logistic: } \begin{cases} \log \left(\frac{1}{1+e^{-w^T h}} \right) & y_i = 1 \\ \log \left(\frac{e^{-w^T h}}{1+e^{-w^T h}} \right) & y_i = 0 \end{cases}$$

$$\text{Multinomial: } \log \left(\frac{e^{w_{y_i}^T h}}{\sum_j e^{w_j^T h}} \right)$$

We can generally design other loss functions for interesting problems. A concern to keep in mind is that the loss function doesn't get too flat (think of a step function), so that it is easy to optimize using gradients.

Regularization

In real problems, you're going to be looking at problems with many millions of parameters.

We definitely need regularization to avoid overfitting!

One approach is to incorporate regularization on the weights directly into the loss. One might add on

$$\lambda \left(\|W^{(1)}\|_2^2 + \|W^{(2)}\|_2^2 \right)$$

to the penalty to help regularize all the layer weights.

These L_2 (ridge) penalties are most common, but L_1 (lasso) penalties are also used for sparse weights.

Note: In practice, penalties are often incorporated in constraint form instead of added onto the loss directly (lagrange form)

Optimization

Now you're ready! We have an objective function, say

$$\|y - \hat{y}\|^2 + \lambda \sum_{k=1}^K \|W^{(k)}\|_2^2$$

We have a prediction function, say

$$\begin{aligned}\hat{y}(x) &= w^T g \left(W^{(2)} g \left(W^{(1)} x + b_1 \right) + b_2 \right) + b \\ &= w^T \max \left\{ 0, W^{(2)} \max \left\{ 0, W^{(1)} x + b_1 \right\} + b_2 \right\} + b\end{aligned}$$

How do we minimize the loss? Is this convex? Can we write the solution in closed form?

How do you feel about training with 10,000,000 samples?

Gradient descent

The neural network problem is non-convex and has many local minima and saddle points. The solution has no closed form.

The most common approach to this optimization problem is a variation of *gradient descent*.

Suppose we want to minimize $F(x)$, $x \in \mathbb{R}^p$. We could compute the gradient at the current point, $x^{(k)}$

$$\nabla F(x^{(k)}) = \left(\frac{\partial F}{\partial x_1}, \dots, \frac{\partial F}{\partial x_p} \right) \bigg|_{x=x^{(k)}}$$

Gradient descent

$$\nabla F(x^{(k)}) = \left(\frac{\partial F}{\partial x_1}, \dots, \frac{\partial F}{\partial x_p} \right) \bigg|_{x=x^{(k)}}$$

The gradient points in the local direction of steepest increase. To walk down hill, we would take a step in the opposite direction:

$$x^{(k+1)} = x^{(k)} - \eta \nabla F(x^{(k)})$$

The step size, η , is also called the *learning rate*. Picking a good value of η is important and there are a variety of approaches.

Gradient descent

Of course, actually computing the gradient is expensive! Our F is the loss, possibly (ignoring regularization):

$$\sum_{i=1}^n (y_i - \hat{y}(x_i))^2$$

The first sum might have 10,000,000 terms! The $\hat{y}$ function might have millions of parameters!

We look for a compromise to carry out gradient descent quickly. This involves two classic tricks.

Stochastic gradient descent

Our gradient based on all the data points is way too expensive to compute. However, what is our gradient?

$$\frac{\partial}{\partial \theta} \sum_{i=1}^n (y_i - \hat{y}(x_i))^2 = \sum_{i=1}^n \frac{\partial}{\partial \theta} (y_i - \hat{y}(x_i))^2$$

This is just an average!

If we use a smaller sample of data points, we'll just get a noisy estimate of this gradient! The noise can even help optimization!

This is what people do in practice.

- ▶ Just evaluate for one point: stochastic gradient descent
- ▶ Evaluate for a small group of points: batch stochastic gradient descent (most common now)

Computing gradients

Great, now we just want to use the gradient for a small set of points.

Are you looking forward to computing the gradient of

$$\sum_{i=1}^m \|y - \hat{y}\|^2 + \lambda \sum_{k=1}^K \|W^{(k)}\|_2^2,$$

$$\begin{aligned}\hat{y}(x) &= w^T g \left(W^{(2)} g \left(W^{(1)} x + b_1 \right) + b_2 \right) + b \\ &= w^T \max \left\{ 0, W^{(2)} \max \left\{ 0, W^{(1)} x + b_1 \right\} + b_2 \right\} + b\end{aligned}$$

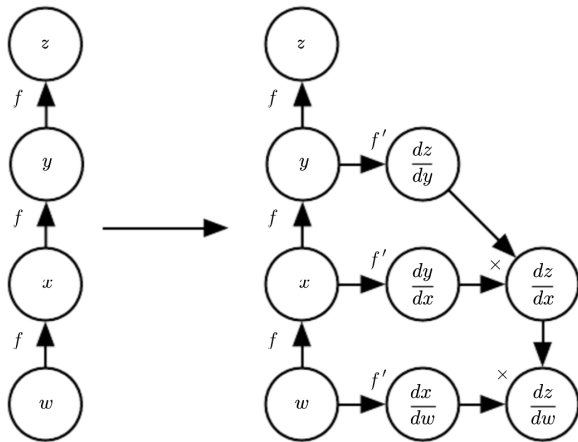
with respect to all the elements of $W^{(1)}$ and $W^{(2)}$?

Backpropagation

We can write out all of our computations as a graph (because the network is nice) and then work our way backwards using the chain rule.

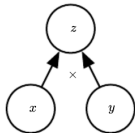
$$\frac{\partial z}{\partial w} =$$

Backpropagation

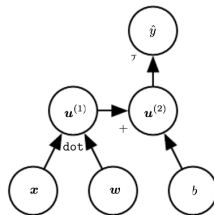


DLB Figure 6.10

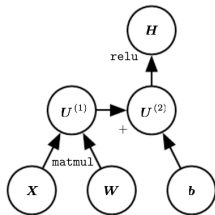
Backpropagation



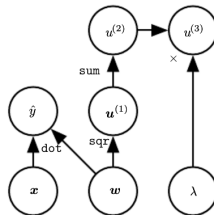
(a)



(b)



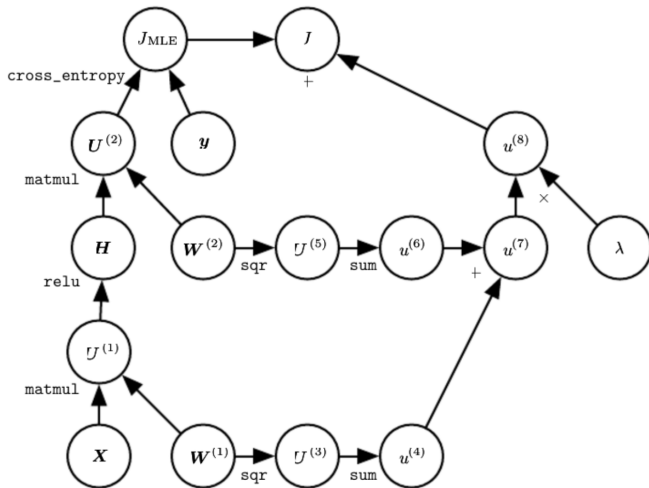
(c)



(d)

DLB Figure 6.8

Backpropagation



DLB Figure 6.11

Beyond backpropagation

Backpropagation is one example of a general strategy called **automatic differentiation**. Notice that:

- ▶ Backpropagation doesn't involve numerical approximations, like the secant method
- ▶ It requires recording the operations applied in each layer as data passes through the network, then applying the chain rule backwards (hence called "reverse mode")
- ▶ It gets *all* the partial derivatives in one pass through the graph

Automatic differentiation isn't just for neural networks: it can be useful any time you need derivatives of complicated functions

Cool tricks: Drop out

We saw in random forests that subsampling can actually improve stability. A similar effect has been observed in deep learning and has been used to great effect.

At each iteration, some nodes (or edges) are hidden

The final network is essentially the average of many simpler networks, but at *reduced* computational cost!

This tends to improve stability and predictions.

Cool tricks: Early stopping

In optimization, we're accustomed to knowing the function that we are optimizing, and we try to seek the local minimum.

In ML (and here), we're optimizing a proxy function! We don't really care about the exact minimum of this function. In fact, the exact minimum usually overfits!

In deep learning, it is customary to stop before the algorithm converges. Various rules exist to guess when all the generalization error gains have been achieved.

Early stopping is cheaper computationally. It acts as a form of regularization, and actually improves predictions!

Cool tricks: Data augmentation

When we talked about the digits data, you might have wondered whether we could just generate new data by transforming our existing images: translate, rotate, thicken them a little.

This is very common in deep learning! It provides a larger training set, and also automatically incorporates invariances.

Cool tricks: Autoencoders

Because of the non-convexity of the problem and all the difficulties with optimization, the initialization network for deep learning is very important.

At the least, the initialization must break ties between neurons.

Giving bigger hints leads to better networks. Many methods exist for coming up with a good starting point.

Most of these tricks rely on the idea that early stages of the network are somewhat task independent! They really capture good representations for the general type of data!

Cool tricks: Autoencoders

An auto-encoder tries to learn the identity function for your X (ignoring any y values).

This seems silly, but is actually non-trivial! What if we don't let the network have enough hidden nodes to actually represent the identity?

The network has to learn an efficient representation of the data to succeed! This can be used to initialize networks for other tasks!

Cool tricks: Autoencoders

Note: There has been quite a bit of recent work showing approaches to fitting neural networks without needing to initialize with autoencoders.

However, they are still useful for

- ▶ Dimension reduction / building representations
- ▶ Denoising
- ▶ Constructing generative models (more to come)

Other architectures

So far, we've been talking about general feed-forward networks with dense connections.

In reality, many different architectures are used for neural networks representing different kinds of problems.

We will briefly introduce the two most common:

- ▶ Convolutional neural networks: For regular, grid-like inputs where we want to share some of the basic processing and use local information.
- ▶ Recurrent neural networks: Sequential data

Convolutional neural networks (CNN)

Suppose you are processing images. The early stages of processing are likely similar everywhere on your image: edge detectors, corner detectors, smoothers, etc.

Furthermore, early processing probably shouldn't depend on long-range relationships across your image. I don't need to see the other corner of an image to decide if I'm looking at an edge.

Convolutional neural networks encode these ideas in their architecture.

Convolutional neural networks (CNN)

The basic idea is to use a small, repeated network to first process the inputs. The weights in the small network are the same at all locations.

These small networks extract interesting local features.

Convolutional neural networks on pictures

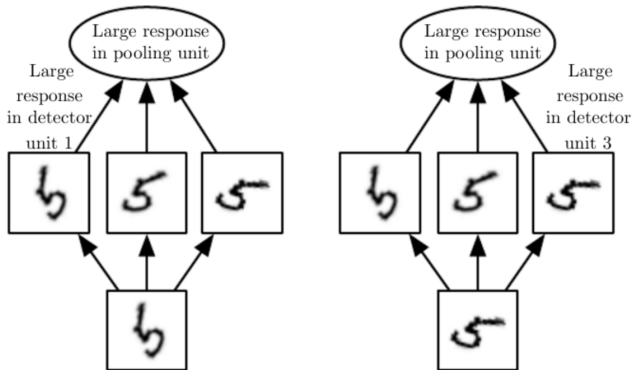
Convolutional neural networks (CNN): Pooling

CNNs usually have one more step. After collecting the local weights and passing them through something non-linear, the results from several features are *pooled* (often max-pooling, which just takes the maximum value).

If we pool neighboring features, we get invariance to exact location.

If we pool different features at the same location, we get invariance to other transformations.

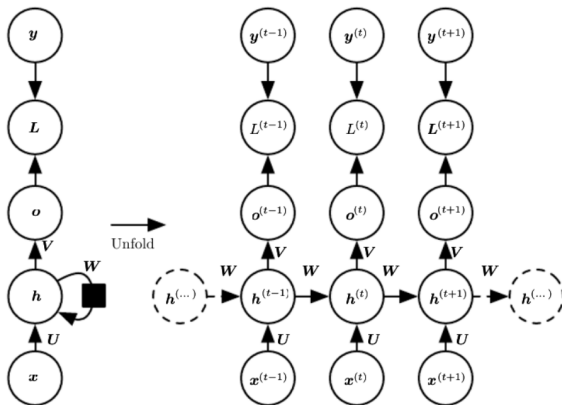
Convolutional neural networks (CNN): Pooling



DLB Figure 9.9

RNN

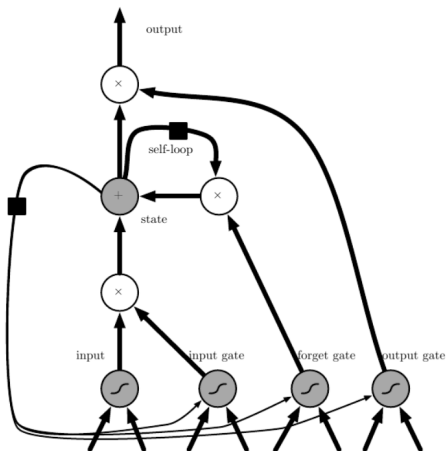
The CNNs work well for tiled data. One of the other common data types is sequential. Recurrent Neural Networks (RNNs) are designed to deal with time-ordered data where we want a sense of memory.



DLB Figure 10.3

RNNs: long-term memory

Many attempts have been made to add long-term memory to RNNs. The most famous are Long Short-term Memory (LSTM) networks.



DLB Figure 10.16

Neural networks as building blocks

Neural networks provide a very flexible class of models, along with tools to successfully train them.

These have been incorporated as flexible functions to solve many other kinds of problem, such as dimension reduction and data generation.

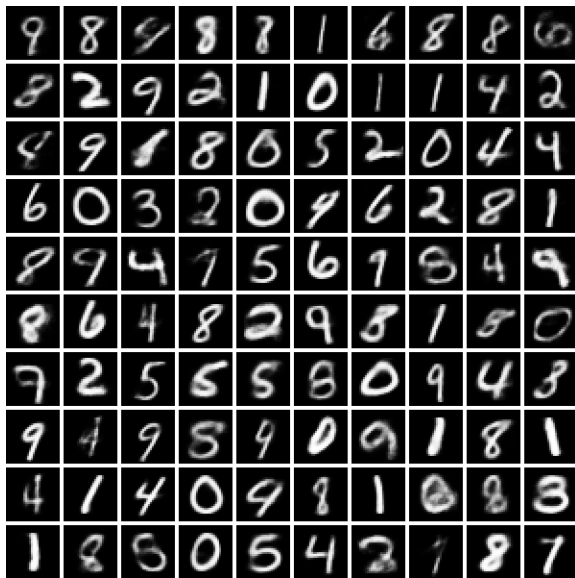
Example: word2vec

<https://jalammar.github.io/illustrated-word2vec/>

Example: Variational autoencoders

<https://www.jeremyjordan.me/variational-autoencoders/>
<https://arxiv.org/abs/1606.05908>

Generating from a variational autoencoder



(from <https://arxiv.org/abs/1606.05908>)

Example: Generative Adversarial Networks (GANs)

<https://www.gwern.net/TWDNE>

Example: Siamese networks

Just a start

This is just a taste of what deep learning looks like. There's a lot of fun to be had learning about this area, but it could (and does) easily fill a whole course.

There are several computing packages now, most of which are Python-friendly. Some of the most popular: Tensorflow, Keras, Torch, Caffe, Theano.